

Question 1 (Multiple Choice)

1. E
2. D
3. C
4. B
5. C
- 6. A or E**
7. B
8. A
9. C
10. E
11. A
12. D
13. D
14. D
15. D
16. B
17. D
18. C
19. C
20. A

Question 2 (Digital Logic)

Q2.1

Q1. An interrupt controller receives interrupt requests (IRQ0, IRQ1, and IRQ2) from three I/O devices. Each device has a different priority, with IRQ0 being the highest and IRQ2 the lowest. The controller has three outputs (OUT0, OUT1, and OUT2), where only one output is asserted (set to Logic 1) at any time, based on the highest active interrupt.

For example:

If IRQ0 = 1, IRQ1 = 1, and IRQ2 = 1, then OUT0 = 1 and OUT1 = OUT2 = 0, since IRQ0 has the highest priority. If only IRQ2 = 1, then OUT2 = 1. If all IRQs are Logic 0, then all outputs are 0.

Breakdown: A = 4, B = 1, C = 2 marks

Q2.1A

A. Draw the truth table for the combinational logic of the interrupt controller.

- Create one version showing all possible input combinations (no simplifications).
- Create a second version using don't-care conditions where lower-priority interrupts can be ignored when a higher-priority interrupt is active.

Part 1 truth table:

IRQ0	IRQ1	IRQ2	OUT0	OUT1	OUT2
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	0
0	1	1	0	1	0
1	0	0	1	0	0
1	0	1	1	0	0
1	1	0	1	0	0
1	1	1	1	0	0

(2 marks) No errors

(1 mark) One cell wrong

(0 marks) Two or more cells wrong

Part 2 truth table:

IRQ0	IRQ1	IRQ2	OUT0	OUT1	OUT2
0	0	0	0	0	0
1	X	X	1	0	0
0	1	X	0	1	0
0	0	1	0	0	1

(2 marks) No errors, or error carried through

(1 mark) One cell wrong, or one don't care reduction omitted

(0 marks) Two or more cells wrong

Q2.1B (1 mark)

- B. Derive the optimized Boolean expressions for each output (OUT0, OUT1, OUT2) using the simplest sum-of-products (SOP) form. Show how each expression is obtained from the truth table.

Solution:

OUT0 = IRQ0

OUT1 = $\sim$ IRQ0 & IRQ1

OUT2 = $\sim$ IRQ0 & $\sim$ IRQ1 & IRQ2

(1 mark) Correct

Q2.1C (2 marks)

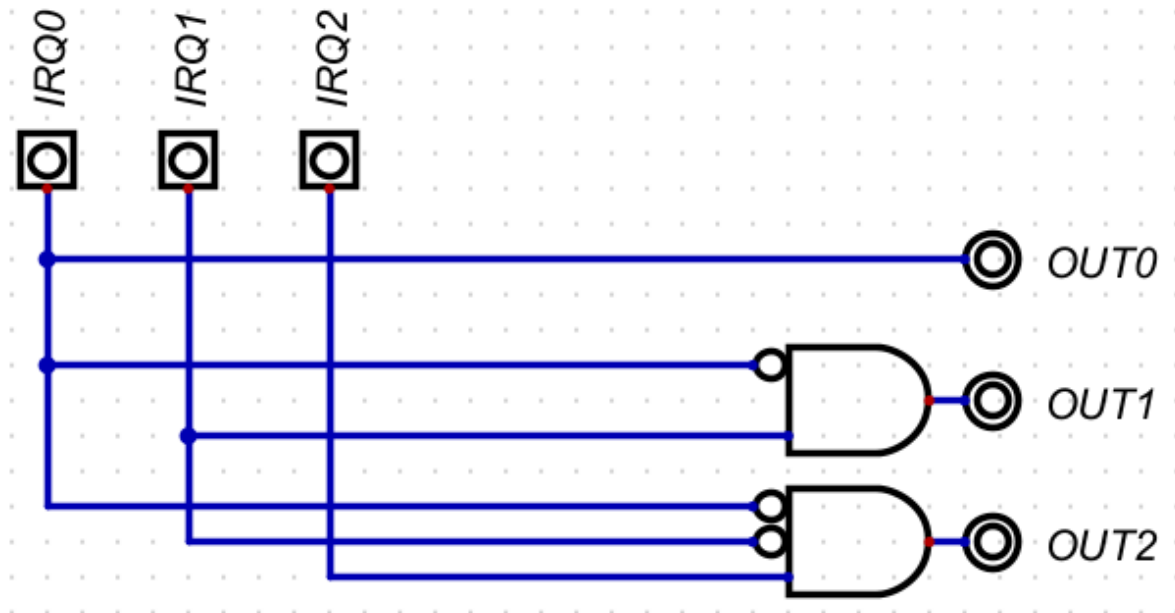
- C. Using only basic logic gates (AND, OR, NOT), draw a logic diagram (schematic) of the interrupt controller. Clearly label all inputs and outputs.

(2 marks) Correct

(1 mark) One mistake (wrong gate or wiring)

(0 marks) More than one mistake

Solution:



Q2.2

Q2. Implement the function shown in the following truth table using a 2:1 multiplexer and two other 2-input logic gates. A, B, and C are inputs, and Y is the output.

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

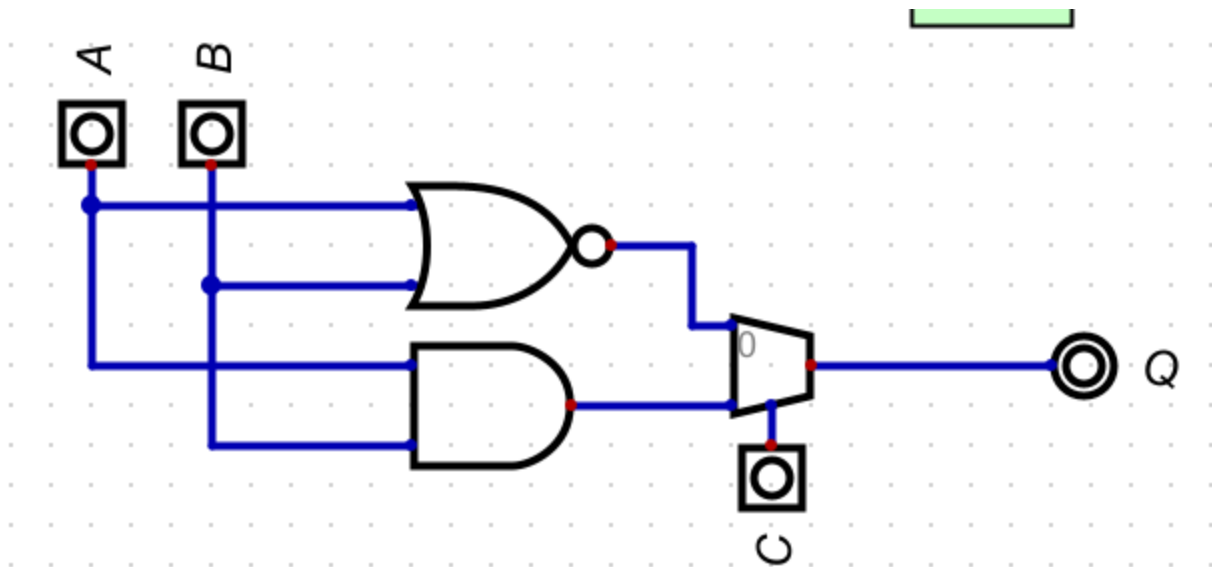
Draw the schematic below.

(3 marks) Correct solution using one multiplexer and two gates

Deduct one for each of these errors: flipped multiplexer, using bubble notation for inverting gate inputs.

Any other mistake/deviation incurs 0 for the question (eg using >2 or incorrect gates).

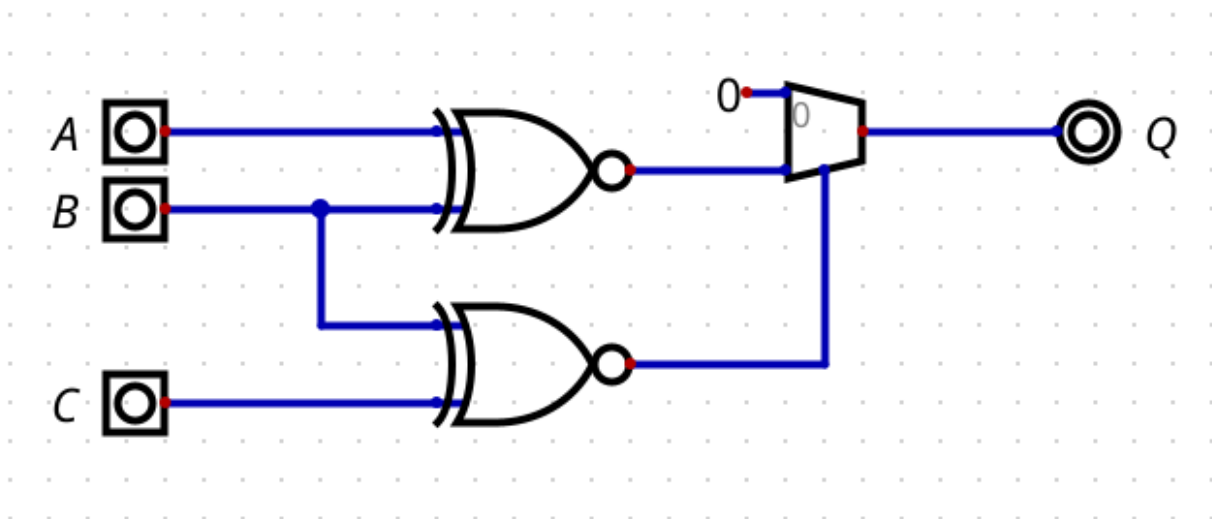
Canonical solution:



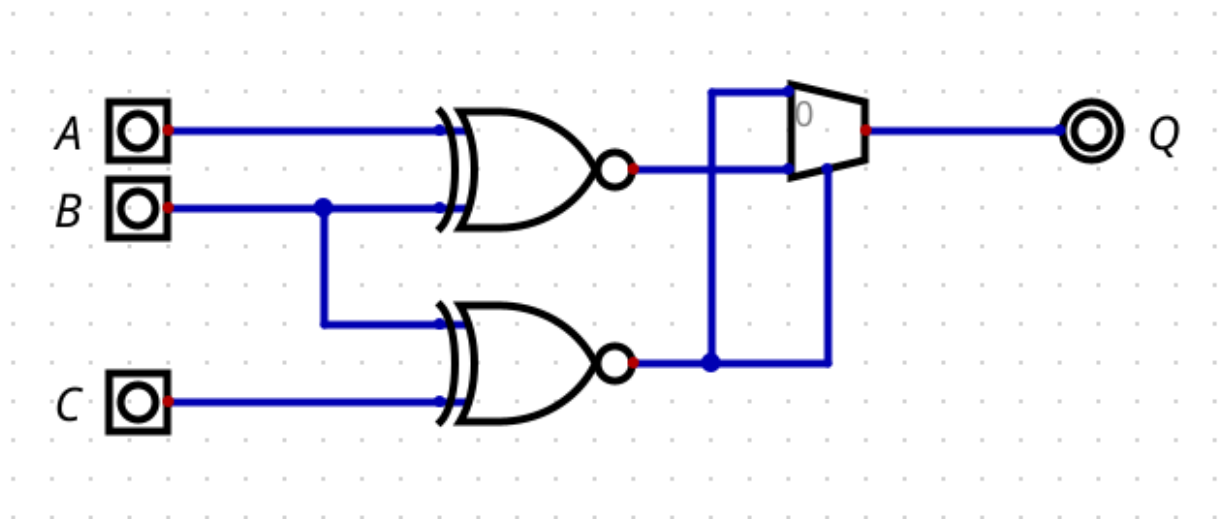
Any version of this with A, B and C swapped around is also correct.

NOTE: there are correct solutions that are completely different structure; see below.

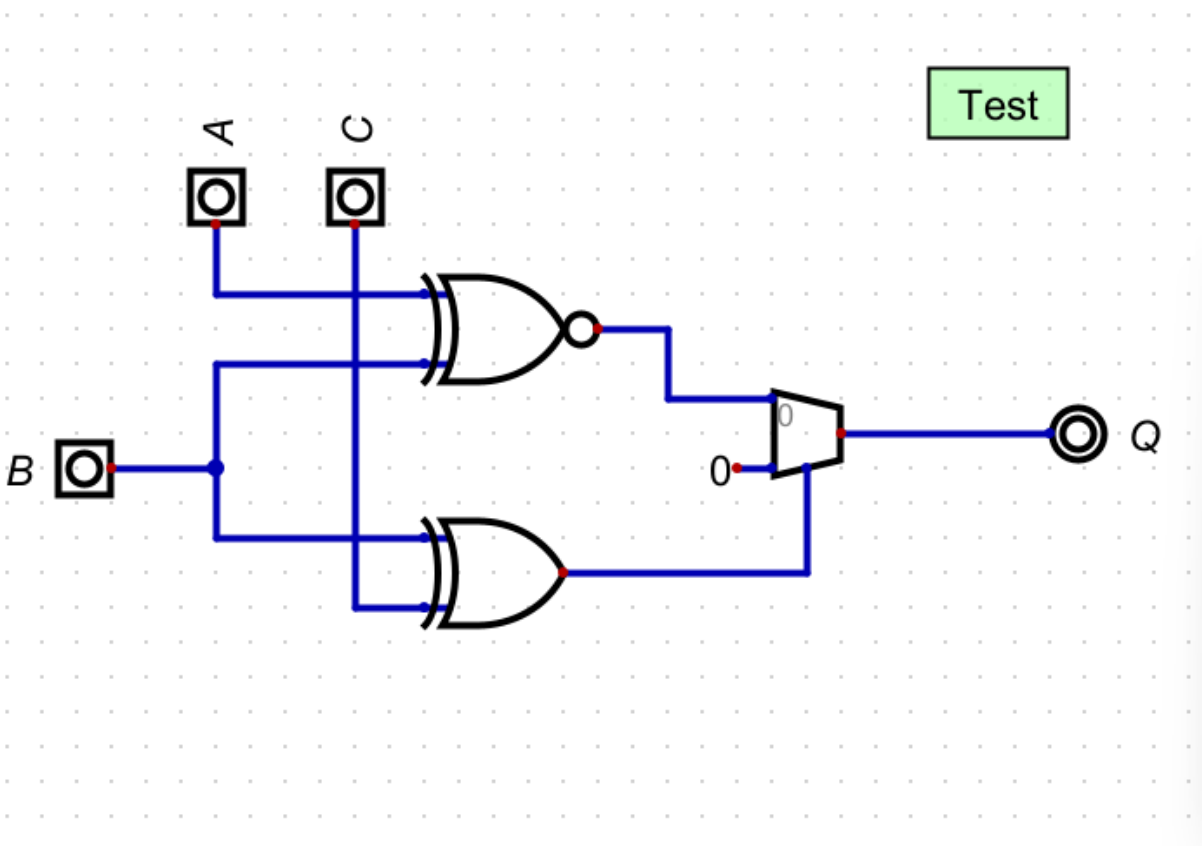
Solution 2a: (relying on using XNOR to check equality of two values)



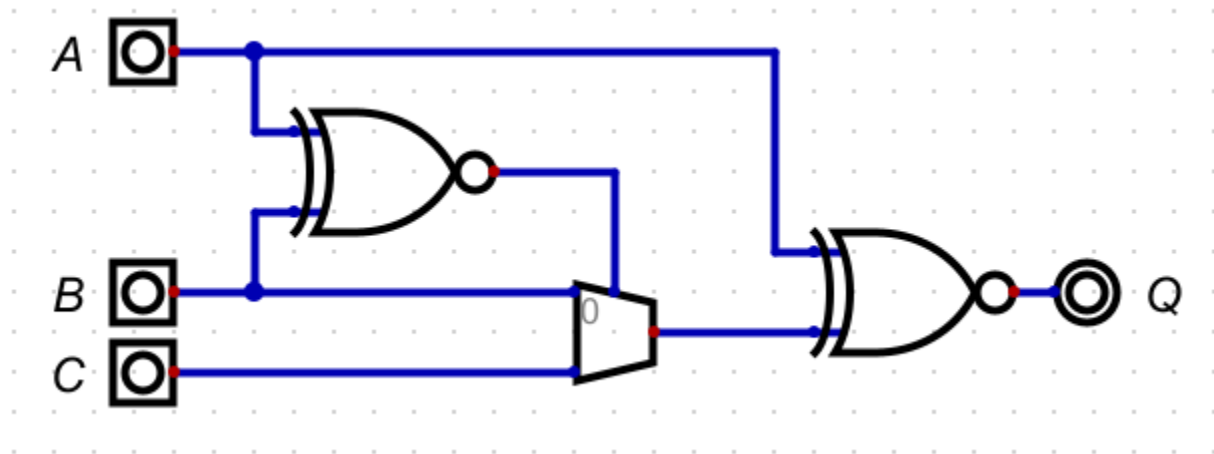
Solution 2b (slight refactoring of 2a)



Solution 3 (same idea as 2 but different formulation)



Solution 4 (same idea as 2 and 3 but different formulation)



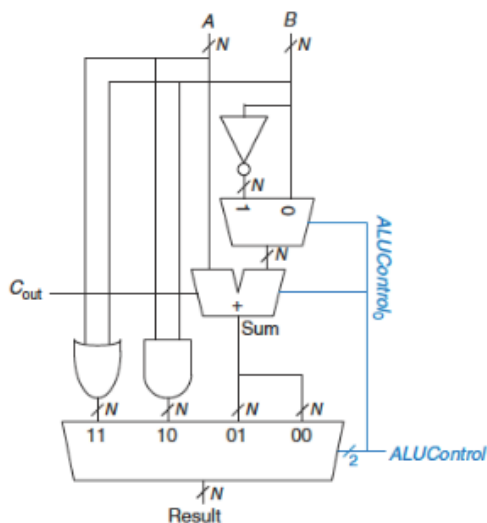
Give more partial marks if only mistake is:

- Using NAND instead of NOR
- (If they do alt solution) using XOR instead of XNOR or vice versa

Question 3 (Timing Issues)

Q3.1

Q1. Consider the schematic of an N-bit ALU for performing two's complement arithmetic below. The ALU contains an N-bit adder and N two-bit AND and OR gates. A 4:1 multiplexer chooses the desired function based on ALUControl. More specifically, if ALUControl is 00, the output multiplexer chooses $A+B$, $A-B$ if 01, $A \text{ AND } B$ if 10, and $A \text{ OR } B$ if 11. The inverter and multiplexer invert input B when ALUControl_0 is asserted (i.e., subtract). Recall that in two's complement arithmetic, $B' + 1 = -B$. Note that picoseconds = ps.



Q3.1A

- A. If the N-bit inverter, the N-bit 2:1 mux, and N-bit adder each has a delay of 1.1 ps, and the N-bit AND/OR gates each have a delay of 3.4 ps, then the critical path consists of:
- a. N-bit inverter, 2:1 N-bit mux, N-bit adder, 4:1 N-bit mux
 - b. N-bit AND, 4:1 N-bit mux
 - c. 4:1 N-bit mux
 - d. 2:1 N-bit mux, N-bit adder, 4:1 N-bit mux
 - e. None of the above is correct

Solution: B (1 mark for correct, 0 for incorrect)

Q3.1B

- B. If N is 32, and the N-bit adder is built using 32 1-bit full adders, where the delay of each one of the 1-bit full adders is 0.25 ps, and all other N-bit components each have a delay of 0.2 ps, then the minimum time to compute $A - B$ is:
- a. 16.4 ps
 - b. 8.2 ps
 - c. 16.2 ps
 - d. 8.4 ps
 - e. 8.6 ps

Solution: E (2 mark for correct, 0 for incorrect)

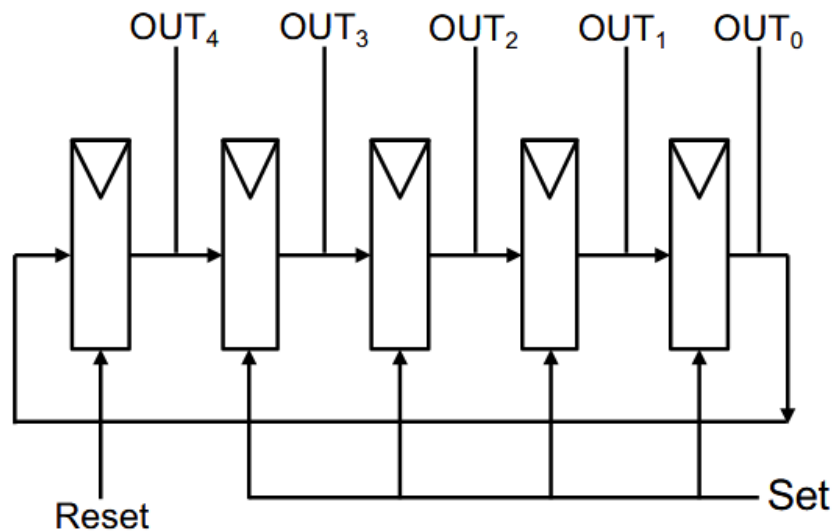
Q3.1C

- C. If $N = 4$, $ALUControl = 01$, $A = 1111$, and $B = 0001$, then $A - B$ is:
- a. 1
 - b. 2
 - c. -1
 - d. -2
 - e. 0

Solution: D (1 mark for correct, 0 for incorrect)

Q3.2

Q2. The following synchronous sequential circuit consists of two types of (rising) edge triggered flip-flops. The left-most flip flop is synchronously resettable (when Reset is Logic 1, the state of the flip-flop is Logic 0 at the next rising edge of the clock; otherwise, input flows to the output at the rising edge). The remaining flip-flops are synchronously settable (when Set is Logic 1, the state of the flip-flop is Logic 1 at the next rising edge of the clock; otherwise, input flows to the output at the rising edge). All flip-flops receive the same clock (CLK) input (not shown). The circuit has a 5-bit output $OUT_{4:0}$.



Timing specification

- The clock-to-Q propagation delay (t_{pcq}) is equal to 0.3 nanoseconds.
- The setup time (t_{setup}) is equal to 0.2 nanoseconds.
- The hold time (t_{hold}) is zero.
- Assume wires have no delay.

Q3.2A

A. Briefly explain what does the above circuit do?

(2 marks) Identifying/describing a bit-shifting operation

(1 mark) Describe the computation correctly ('moves' the number across), but do not identify shifting.

Q3.2B

B. What is the maximum clock frequency at which a circuit designer can operate the circuit?

(2 marks) Correct answer (2 GHz)

(1 mark) One mistake (single arithmetic error, single incorrect value, forget one of the values) OR correct answer but in ns/time(0.5ns)

(0 marks) Incorrect with more than one error.

Q3.2C

C. Assume set/reset are asserted just prior to the rising edge of the clock, respecting the setup constraint. What is the delay (in seconds) for $OUT_{4:0}$ to transition from 01111 to 11110 (beginning at the start of the cycle, i.e., just after the rising edge)?

(2 marks) Correct answer, or error carried through from previous (no working required, 4 cycles at clock rate, should be 2 nanoseconds if B correct)

(1 mark) Gave answer in cycles rather than seconds

(0 marks) Incorrect result (wrt error carried through), invalid justification for answer (e.g. '8 cycles at a clock of 4 GHz or something)

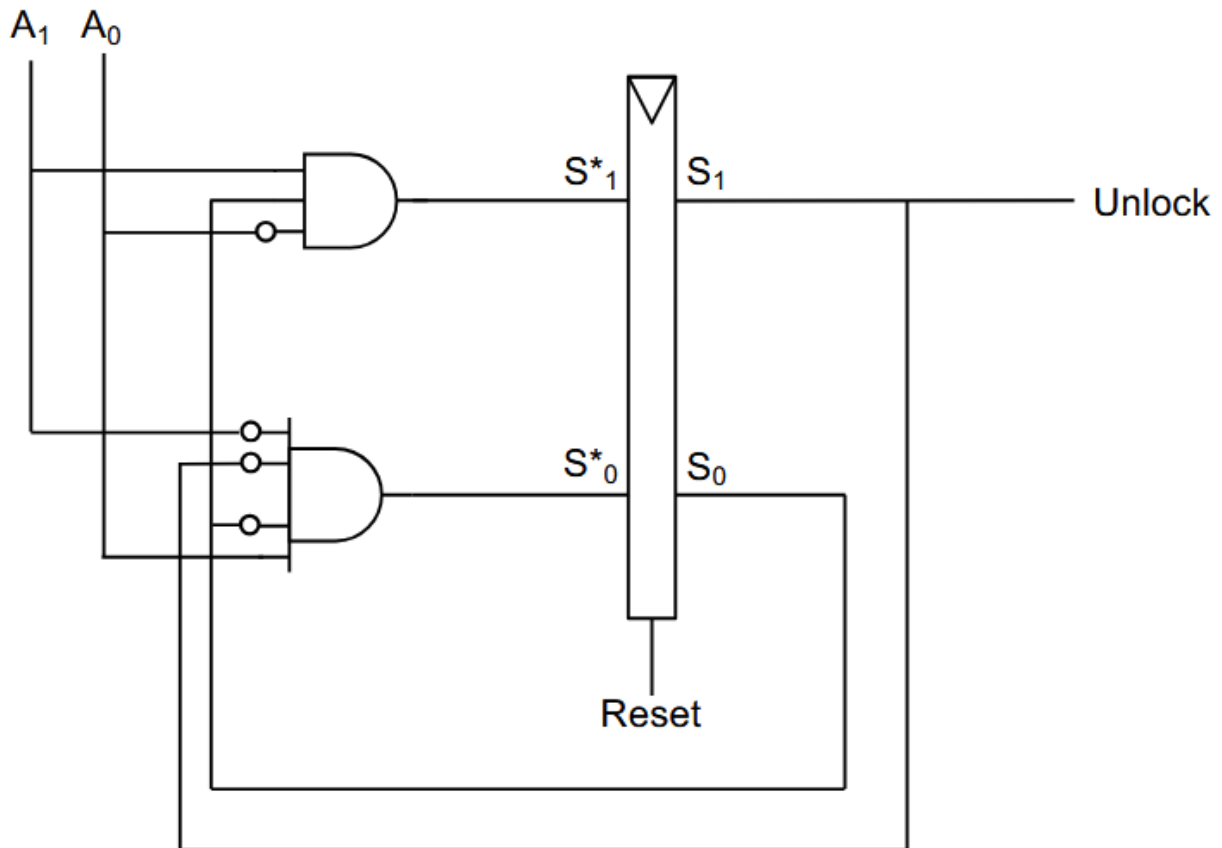
(Mark 5 cycles instead of 4 as 1 mark instead of 0, and the same for if they add startup time).

Question 4 (Finite State Machine)

ERROR CARRIED THROUGH IN ALL PARTS

Q4.1

Q1. The circuit below implements a Moore finite state machine (FSM). The 2-bit current state is given by bits S_0 and S_1 . The next state is given by bits S^*_0 and S^*_1 . The output of the circuit is Unlock, and the external inputs are A_0 and A_1 .



Q4.1A (3 marks)

A. Write the next state and output equations.

1 mark per correct equation.

Solution:

$$S_0^* = \bar{S}_0 \bar{S}_1 A_0 \bar{A}_1$$

$$S_1^* = S_0 \bar{A}_0 A_1$$

$$Unlock = S_1$$

Q4.1B (4 marks)

B. Write down the encoded output and state transition tables. Use binary encoding for representing states. Use reduction using don't cares whenever possible.

(4 marks) Perfect state and output tables with all reductions.

-1 for missing each of the two groups of required reductions in Solution 1 below (i.e. 2 marks for perfect but non-reduced table.) That is, the 4 reductions in the blue rows and the reduction in the purple row. Can be reduced further to Solution 2, but this is honestly more annoying than helpful. (max penalty of -2 for unreduced tables)

-1 per row with any incorrect cells in the state table

-1 if the output table is wrong in any way

If they have done the unreachable states, and crossed them out, do not penalize (the cross out indicating them being unreachable).

If they forget to include the 11 states at all, and later write that they aren't reachable in F, then only -1.

Solution 1 (mostly reduced)

A0	A1	S0	S1		S0*	S1*	
0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0
1	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0
0	0	0	1	X	0	0	0
0	1	1	1	X	0	1	1
1	0	1	1	X	0	0	0
1	1	1	1	X	0	0	0
X	X	0	1		0	0	0

Solution 2 (with further reduction, but honestly makes it more messy)

A0	A1	S0	S1		S0*	S1*	
0	X	0	0	0	0	0	0
1	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0
0	0	1	1	X	0	0	0
0	1	1	1	X	0	1	1
1	X	1	1	X	0	0	0
X	X	0	1		0	0	0

Unreduced solution:

A0	A1	S0	S1		S0*	S1*	
0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0
1	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0
0	0	1	1	0	0	0	0
0	1	1	1	0	0	1	1
1	0	1	1	0	0	0	0
1	1	1	1	0	0	0	0
0	0	0	0	1	0	0	0
0	1	0	0	1	0	0	0
1	0	0	0	1	0	0	0
1	1	0	0	1	0	0	0
0	0	1	1	1	0	0	0
0	1	1	1	1	0	1	1
1	0	1	1	1	0	0	0
1	1	1	1	1	0	0	0

Q4.1C (2 marks)

C. Write down the symbolic next state and output tables. Use any naming scheme in English for representing states.

(2 marks) Perfect

(1 mark) Mostly correct (can be fixed by swapping inputs around). If missing only one transition.

(0 marks) ≥ 2 mistakes

IF STUDENTS COMPLETE PART C IN PART B, THEN MARK AS THOUGH PART B

A=A1 A0, S = S1 S0			
A	Current state		Next state
0	S0	S0	S0
1	S0	S0	S1
2	S0	S0	S0
3	S0	S0	S0
0	S1	S1	S0
1	S1	S1	S0
2	S1	S1	S2
3	S1	S1	S0
X	S2	S2	S0

Note: several possible encodings for A and S. Also they don't actually have to encode A.

Q4.1D (3 marks)

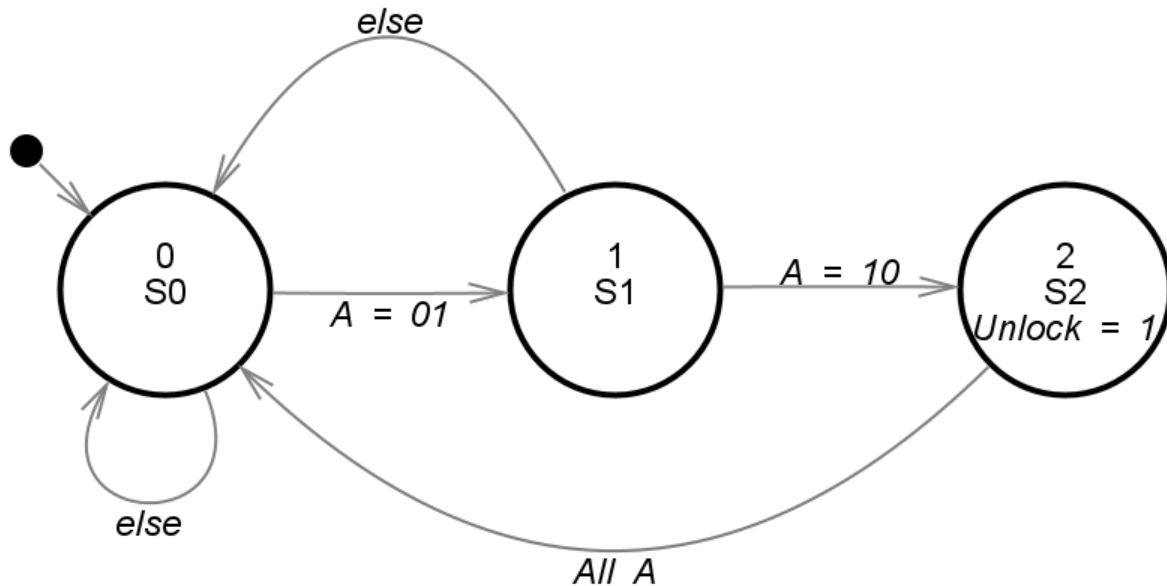
D. Draw the state machine diagram of the above circuit.

(3 marks) All states, transitions, and labels on both correct

-1 mark for each missing/extra/incorrect transition arrow/label

-1 mark for each missing/extra state if S3 is included, and it is also in the truth table, then

Q4.1E (1 marks)



This also depends on encoding.

E. In simple words, briefly explain the purpose of the above state machine.

Exemplar: A passcode/combination lock/etc. mechanism

(1 mark) Any valid explanation (not a description of the machine, an explanation of its high-level purpose). They need to show understanding of the purpose.

Q4.1F (1 marks)

F. Suppose the FSM designer implements the output, Unlock, as the AND of the two state bits, S_0 and S_1 . Explain the behavior of Unlock in this case.

Exemplar: Unlock will never be set to 1 because S_1S_0 is an unreachable combination

(1 mark) Correctly state it will **never unlock**, or that the state is unreachable

No mark awarded if they just say 'oh it now opens in state 11' - they don't demonstrate understanding that it never unlocks / 11 is unreachable.

Question 5 (ARM Assembly)

Q1. Translate the following C functions into 32-bit ARM assembly code. The parameters of all functions are provided in R0 and R1. The result should be returned in R0. Assume the base address of array is 0x0000FFFF. Implement all three functions, and do not use inlining.

Your code should preserve registers using the ARM calling convention, if necessary.

```
// C code
void setArray(int num) {
    int i;
    int array[10];

    for (i = 0; i < 10; i++)
        array[i] = compare(num, i);
}

int compare(int a, int b) {
    if (sub(a,b) >= 0)
        return 1;
    else
        return 0;
}

int sub(int a, int b) {
    return a - b;
}
```

Each sub-question below has the relevant function with its sample solution. The total number of marks for this question is 10, which are distributed as 6 for setArray, 3 for compare and 1 for sub.

Notes about special cases/other calls made when marking will be listed here:

- Some students had loaded from the array/had other additional instructions that weren't needed. If they didn't interfere with the correctness of the rest of the function, that is considered fine.
- Didn't consider it a mistake to push/pop lr in a leaf function (i.e. sub).
- For a case with a student who had the correct idea for which memory address to store to, but chose the wrong instruction (i.e. they did a ldr r4, [r3,r2,lsl #2]) for the

calculation, counted that as a syntax error and gave mark for storing to correct address

- Similar to above: “b lr” rather than “bx lr” counted as mark for returning correctly, but -1 for syntax error.

Q5.1

setArray

```
1  .type setArray, %function
2  @ args:
3  @ r0: num
4  setArray:
5      push {r4, lr}
6      ldr r2, =0xFFFF @ array
7      ldr r3, =0
8      mov r4, r0 @ num
9  setArray_loop:
10     cmp r3, #10 @ array length
11     bge end_setArray @ if i >= 10, done
12     mov r0, r4
13     mov r1, r3
14     bl compare @ r0 := compare(num, i)
15     lsl r1, r3, #2 @ r1 := i*4
16     str r0, [r2, r1] @ array[i] = compare(num, i)
17     add r3, #1
18     b setArray_loop
19 end_setArray:
20     pop {r4, lr}
21     bx lr
22 .size setArray, . - setArray
```

Marking Guide for setArray:

Correctly saves link register and callee-save registers on stack: +1

- If they don't call compare correctly (so it acts as a leaf function) they can still receive the marks for this if they save callee-save registers correctly.

- Returns from function correctly (pops lr, bx lr/pop pc): +1
- Initializes all variables correctly (i is 0, array address is 0xffff): +1
- For loop condition check correct: +1
- Correctly calls “compare” function, including saving required caller-save registers around the call to compare: +1
 - **Note:** will still count cases where caller-save registers aren't saved when they don't end up getting modified in compare or sub, like r3 in my sample solution above.
- Updates correct memory address (i.e. i*4): +1
 - Don't give mark if they forget to increment the loop variable
- If there are any syntax errors/naming mistakes/etc.: -1 mark from the above

- Syntax errors include incorrect instruction mnemonics, using QuAC, incorrect label names used in function calls/branches. Incorrect names include not following the naming scheme we gave them

Q5.2

compare

```

24  .type compare, %function
25  @ args:
26  @  r0: a
27  @  r1: b
28  compare:
29      push {lr}
30      bl subtract
31      cmp r0, #0
32      bge if_ge_zero
33      mov r0, #0
34      b end_compare
35  if_ge_zero:
36      mov r0, #1
37  end_compare:
38      pop {lr}
39      bx lr
40  .size compare, . - compare
41

```

Marking Guide:

- Correctly calls ‘sub’ function AND saves link register/other necessary registers on stack: **+1**
- Returns correct value in r0: **+1**
- If/Else Implemented Correctly: **+1**
 - Either using labels and conditional jumps or IT block, but it has to be correct either way. Conditional mov without being in an IT block does not count.
- If they inline the function but everything else is correct, only give **1 mark** for this part. Should still have a label “compare” or equivalent, though.
- If there are any syntax errors/naming mistakes/etc.: **-1 mark from the above**
 - Syntax errors include incorrect instruction mnemonics, using QuAC, incorrect label names used in function calls/branches. Incorrect names include not following the naming scheme we gave them
 - Don’t deduct if syntax errors are the same as ones made in setArray (e.g. push/pop without curly brackets is the same mistake in both).

Q5.3

sub

```
42  .type subtract, %function
43  @ args:
44  @ r0: a
45  @ r1: b
46  subtract:
47      sub r0, r0, r1
48      bx lr
49  .size subtract, . - subtract
50
```

Note: just to make the syntax highlighting consistent, “subtract” was used for the function name rather than “sub”. Don’t consider renaming this function a mistake.

Marking Guide:

- **Fully correct: +1**
 - This means *any* mistakes (returning in wrong register, not bx lr’ing, incorrect registers used as arguments) mean they don’t get the mark.

Question 6 (Input/Output)

Q6.1

Q1. A 128 KB Flash memory-based storage device has a built-in cache for storing the most recently used data. The cache can be enabled or disabled by writing to a memory-mapped control register on the device. The device access delays are as follows:

- A hit costs one clock cycle.
- A miss costs 4 clock cycles.
- A device access if the cache is disabled costs 3 clock cycles.

Is it beneficial to enable the cache for the hit rates below? Show your analytical reasoning and write Yes or No below. A simple Yes or No without reasoning provided will lead to 0 marks.

Q6.1A (1.5)

A. 25%: _____

Correct answer: No, $(\frac{1}{4} \cdot 1 + \frac{3}{4} \cdot 4) \rightarrow 3.25 > 3 \rightarrow$ no

Q6.1B (1.5)

B. 50%: ____

Correct answer: Yes, ($\frac{1}{2} \times 1 + \frac{1}{2} \times 4$) $\rightarrow 2.5 < 3 \rightarrow$ yes

For both:

1 mark added for correct reasoning/equations

0.5 marks added if they state 'yes' or 'no' correctly with that working

Q6.2

Q2. An embedded system uses memory-mapped I/O to interact with peripheral devices. A sensor value can be read (either 0 or 1) from the memory-mapped address 0x40000000. An LED can be turned on by setting bit 8 of a control register at address 0x30000000.

Write 32-bit ARM assembly code to continuously poll the sensor until it reads Logic 1. Then, set bit 8 of the LED control register to turn the LED on. Exit the loop after setting the LED.

(3 marks) No mistakes

Deduct 1 mark per line that contains a mistake, or is invalid ARM

Deduct 1 mark if the loop logic is incorrect

Deduct 1 mark if they overwrite the register rather than load/twiddle/store

(If people shift by 8 instead of 7, will choose not to deduct this mark).

The constants can be put into register both with mov and ldr =; but if they do ldr without equals, i.e. ldr r0, 0x40000000, this is wrong. Example solution:

```
ldr r0, =0x40000000 @ or 'mov 0x40000000'
ldr r1, =0x30000000 @ or 'mov 0x30000000'
check:
ldr r2, [r0]
cmp r2, #1
beq end
loop:
b check
end:
mov r3, #0x1 @ note that these two lines can be done in one line
lsl r3, #7
ldr r2, [r1]
orr r2, r2, r3
str r2, [r1]
```

Another example solution:

```
ldr r0, =0x40000000
```

```
loop:
```

```
ldr r1, [r0]
```

```
cmp r1, 1
```

```
bne loop
```

```
ldr r0, =0x30000000
```

```
ldr r1, [r0]
```

```
orr r1, (1 << 7)
```

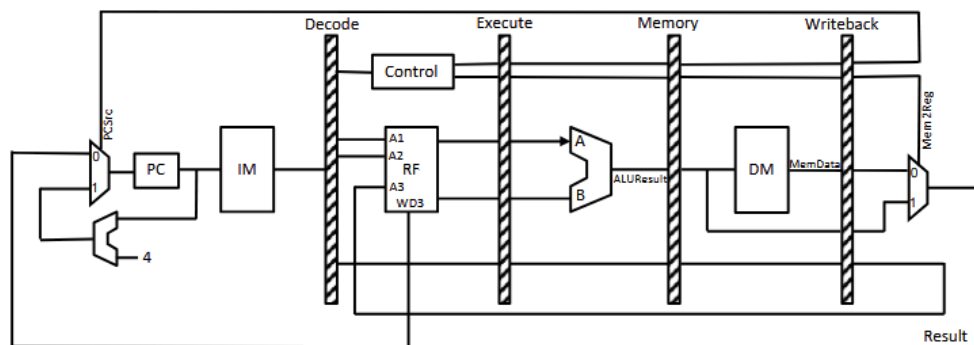
```
str r1, [r0]
```

```
@ exit
```

Question 7 (Microarchitecture)

Q7.1

Q1. Consider the *scalar* in-order pipelined CPU shown below. The five pipeline stages are Fetch, Decode, Execute, Memory, and Writeback. Each stage performs its work in one clock cycle. The pipeline registers are labeled with the stage they are in. PC is in fetch stage. The CPU lacks forwarding and hazard detection capabilities, including no ability to deal with incorrect branch prediction. ALU operations and memory accesses take one clock cycle. The address of a memory instruction is computed in the Execute stage. The register file is written during the first half of the clock cycle, and read during the second half of the clock cycle.



For the instruction sequence shown below, rewrite the assembly code by inserting NOPs to execute the code correctly on the pipelined CPU. Your answer is only valid if you use as few NOPS as possible. Do not write complete instructions. Use instruction labels, i1, i2, etc.

```
i1: LDR R3, [R2, #4]
i2: ADD R4, R3, R2
i3: ADD R5, R4, #1
i4: SUB R7, R5, R5
i5: B TARGET
i6: ADD R0, R0, #1
i7: ADD R1, R1, #1
    TARGET:
i8: ADD R8, R0, #1
i9: ADD R9, R1, R5
i10: STR R9, [R12]
```

Exemplar:

i1, 2 nops

i2, 2 nops

i3, 2 nops

i5, 4 nops

i9, 2 nops

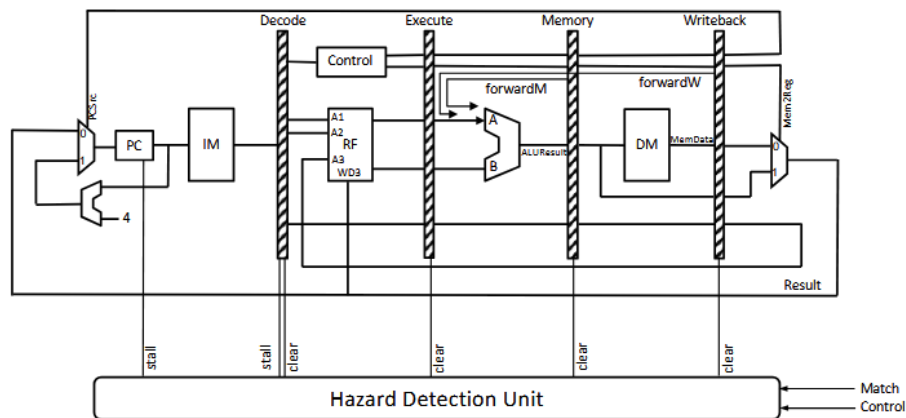
(4 marks) Correct

-1 per incorrectly (added/missed) position block of NOPs

0.5 per incorrect count in a correctly positioned block of NOPs

Q7.2

Q2. Consider the pipelined CPU from Q1, but now, with forwarding and hazard detection in hardware. Assume in the case of always-untaken prediction policy, the CPU is capable of flushing pipeline registers on misprediction. Also, in the case of always-taken branch prediction policy, CPU is capable of stalling the pipeline as soon as the branch is decoded (i.e., pipeline is stalled when the branch is in the Decode stage). The register file is written during the first half of the clock cycle, and read during the second half of the clock cycle. CMP sets flags in EX, and BEQ reads flags in EX, i.e., CMP-BEQ pair is not a hazard. PC is updated at the rising edge of the clock.



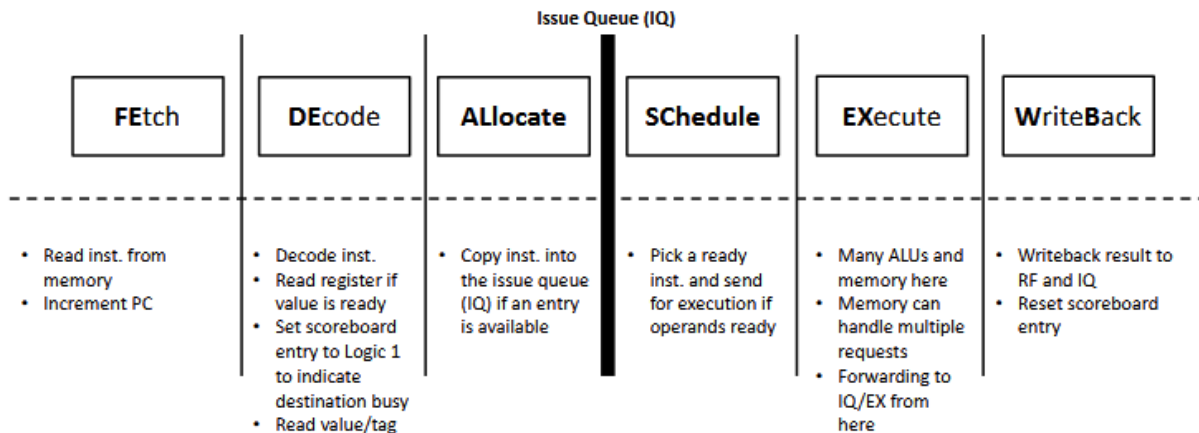
Now, carefully study the following instruction sequence. The address of each instruction is shown in hexadecimal on the left. Each instruction is 32 bits in size.

```
0x00: MOV R0, #0
0x04: MOV R1, #1
      LOOP:
0x08: CMP R0, R1
0x0C: BEQ DONE
0x10: LDR R2, [R6, R7, LSL 2]
0x14: ADD R2, R2, #4
0x18: STR R2, [R6, R7, LSL 2]
0x1C: ADD R0, R0, #1
0x20: B LOOP
      DONE:
0x40: NOP
0x44: NOP
0x48: NOP
0x4C: NOP
```

- A. Assuming CPU uses the branch-always-taken policy for branch instructions, which instruction's address (hexadecimal) does PC contain in cycle # 8: ____
- B. Assuming CPU implements the branch-always-untaken policy for branch instructions, and branches resolve (i.e., update PC with correct target address) in the writeback stage, which instruction's address (hexadecimal) does PC contain in cycle # 8: ____
- C. Assuming CPU implements the branch-always-untaken policy for branch instructions, and branches resolve (i.e., update PC with correct target address) in the execute stage, which instruction's address (hexadecimal) does PC contain in cycle # 8: ____
- D. Which one of the four pipeline registers (see labels in schematic) forwards R2 to instruction at the address 0x14: ____
- E. In which cycle at the rising edge does the partially processed CMP instruction appears in the Memory pipeline register: ____
- F. Assuming CPU implements the branch-always-untaken policy for branch instructions, and branches resolve (i.e., update PC with correct target address) in the writeback stage, in which cycle does PC contains 0x40: ____
- G. Assuming CPU implements the branch-always-untaken policy for branch instructions, and branches resolve (i.e., update PC with correct target address) in the execute stage, how many cycles does it take to process all instructions (including NOPs): ____
- H. What is the output of the 2:1 multiplexer in the fetch stage at the end of cycle #3: ____
- A. 0x10
- B. 0x1C
- C. 0x1C
- D. Writeback
- E. 6
- F. 20
- G. 23
- H. 0x0C

Q7.3

Q3. Consider a dynamically scheduled out-of-order CPU pipeline shown below. The six stages are FE, DE, AL, SC, EX, and WB. Carefully read what each stage does below. DE contains a scoreboard to indicate valid values and busy registers. AL copies instructions into an issue queue (IQ) or scheduler. SC issues (sends) ready instructions to the EX stage. EX contains many functional units as well as memory. The scoreboard has eight entries one corresponding to each register in the hypothetical ARM ISA. Before the execution of the first instruction, all scoreboard entries are set to 1 (i.e., values in RF are ready, and no register is busy). A busy register, i.e., value is being computed, and dependent instruction must wait, is indicated by Logic 0 in a scoreboard entry. The IQ is empty with 16 entries available for allocation.



Q7.3A (6 marks)

A. Complete the pipeline diagram below by indicating, for each cycle, the stage (e.g., FE) that each instruction is in. Assume each memory access takes four cycles to complete.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
i1: LDR R2, [R1, #0]																	
i2: LDR R4, [R3, #1]																	
i3: ADD R5, R2, #1																	
i4: SUB R6, R4, #1																	

	1	2	3	4	5	6	7	8	9	10	11	12
i1	F	D	A	S	E	E	E	E	W			
i2		F	D	A	S	E	E	E	E	W		
i3			F	D	A	A/IQ	A/IQ	A/IQ	S	E	W	
i4				F	D	A	A/IQ	A/IQ	A/IQ	S	E	W

1.5 marks per row correct

Q7.3B (2 marks)

B. What are the contents of the following scoreboard entries during cycle # 9.

R1____
R2____
R3____
R4____

(2 marks) all correct, with either colouring.

Correct answer is Ready, Ready, Ready, Busy, which reads 0001 or 1110.

(If people do 1010/0101 instead of 1110/0001, this can be marked as correct).

Q7.3C (2 marks)

C. List four issues in the out-of-order (OOO) CPU in B above that are addressed by an OOO CPU with register renaming and hardware speculation (one short bullet point per issue).

- WAR hazards
- WAW hazards
- Branch misprediction recovery, repair, mis-speculation
- Precise interrupts/exceptions

0.5 per